Topic 4 | Support Ticket Priority Routing

Assessment evidence report | n8n workflow | 16 October 2026

Purpose

Document the observed configuration and execution of support-ticket validation, priority-based routing, simulated notification dispatch, audit logging and invalid-ticket rejection. The screenshots below are provided as direct evidence, with a short explanation for each.

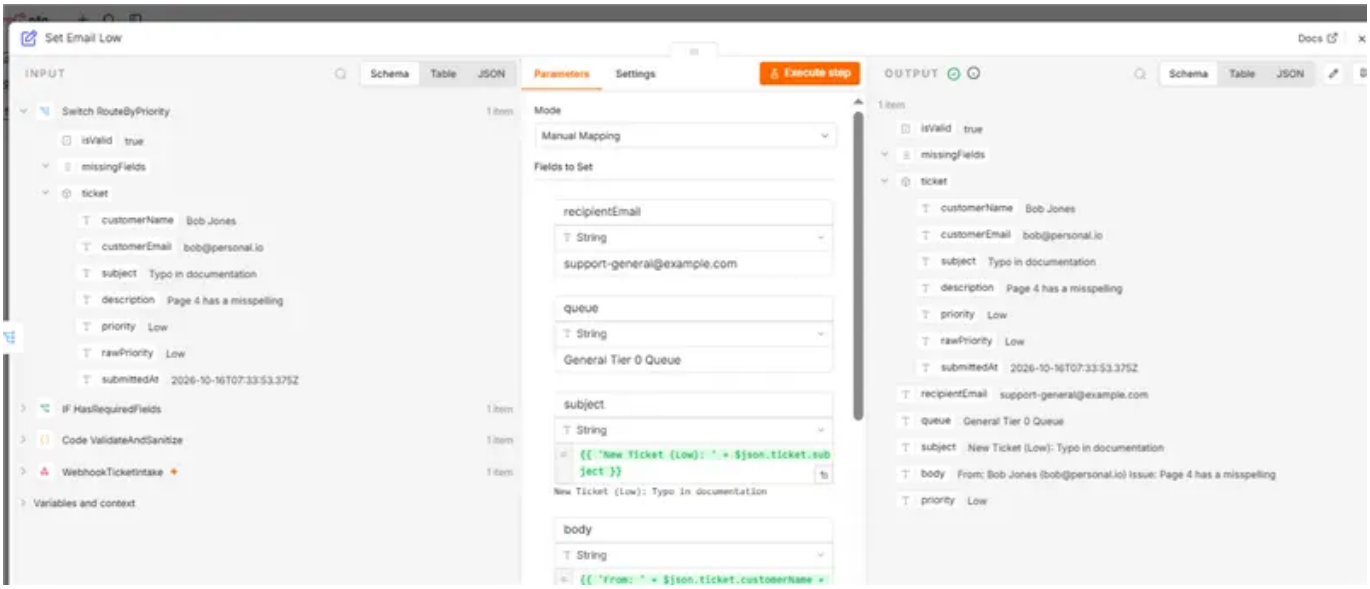
Observed results

Evidence area	Status shown in screenshots
Low and Medium routing	Node outputs verified
Fallback routing (Critical)	Node output and HTTP dispatch verified
Mock notification server	Published; successful execution shown
Routing audit	Successful Fallback audit output shown
Missing-email rejection	Rejection output verified
High and Urgent end-to-end tests	Not evidenced by these screenshots

Scope note

The DELIVERED status comes from a mock response and does not prove that a real email was sent. The screenshots do not establish separate end-to-end results for High and Urgent tickets. These tests should be captured before claiming complete priority coverage.

01 | Low priority notification mapping

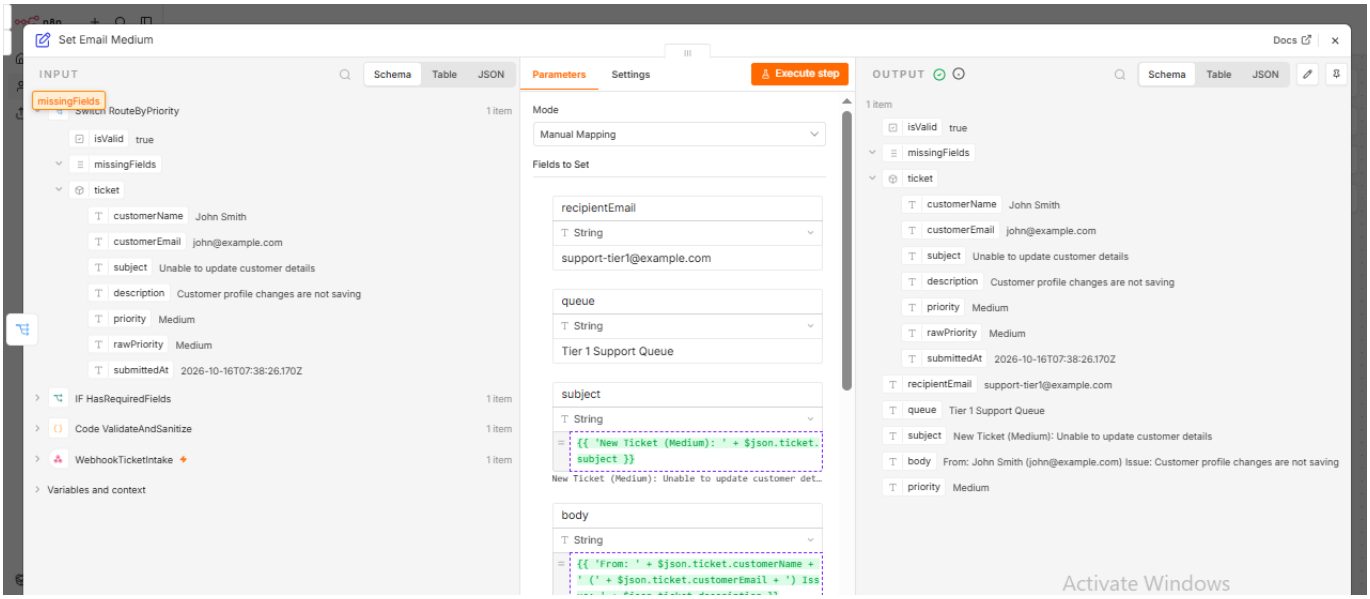


Explanation

The Set Email Low node receives a valid Low-priority ticket and prepares the General Tier 0 Queue recipient, subject and notification body. The output displays the mapped values for Bob Jones.

Source screenshot: image(20261016-144706).png

02 | Medium priority notification mapping

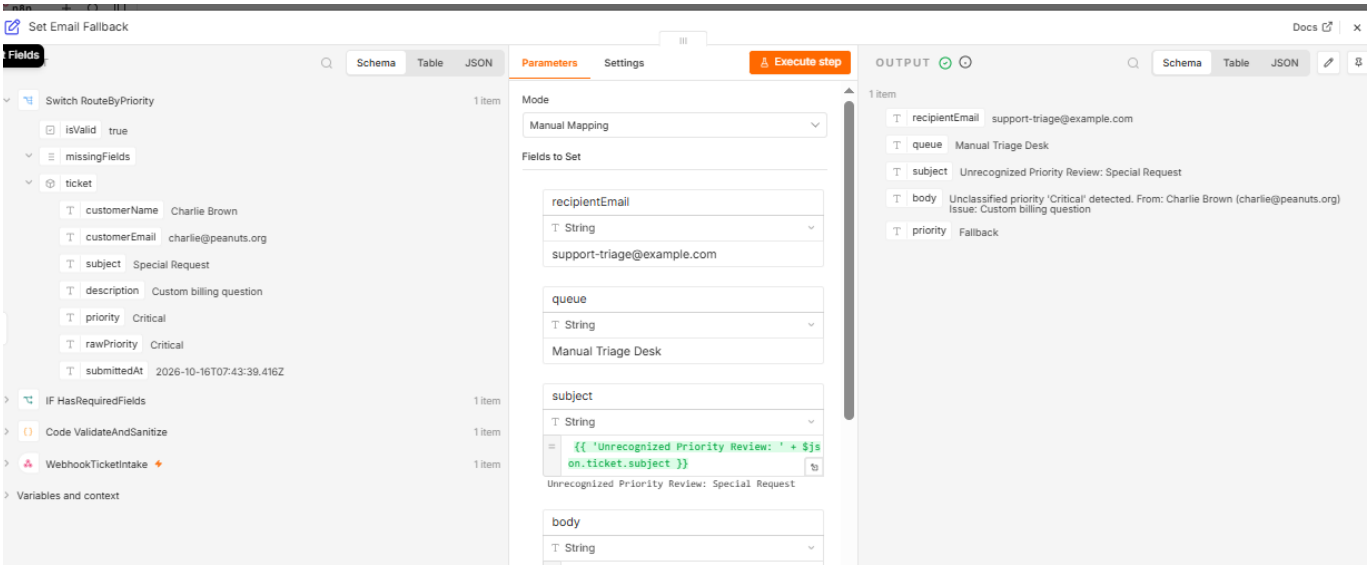


Explanation

The Set Email Medium node routes a valid Medium-priority ticket to the Tier 1 Support Queue. Its output shows the selected recipient and dynamically generated subject and message body.

Source screenshot: image(20261016-144721).png

03 | Unsupported priority fallback

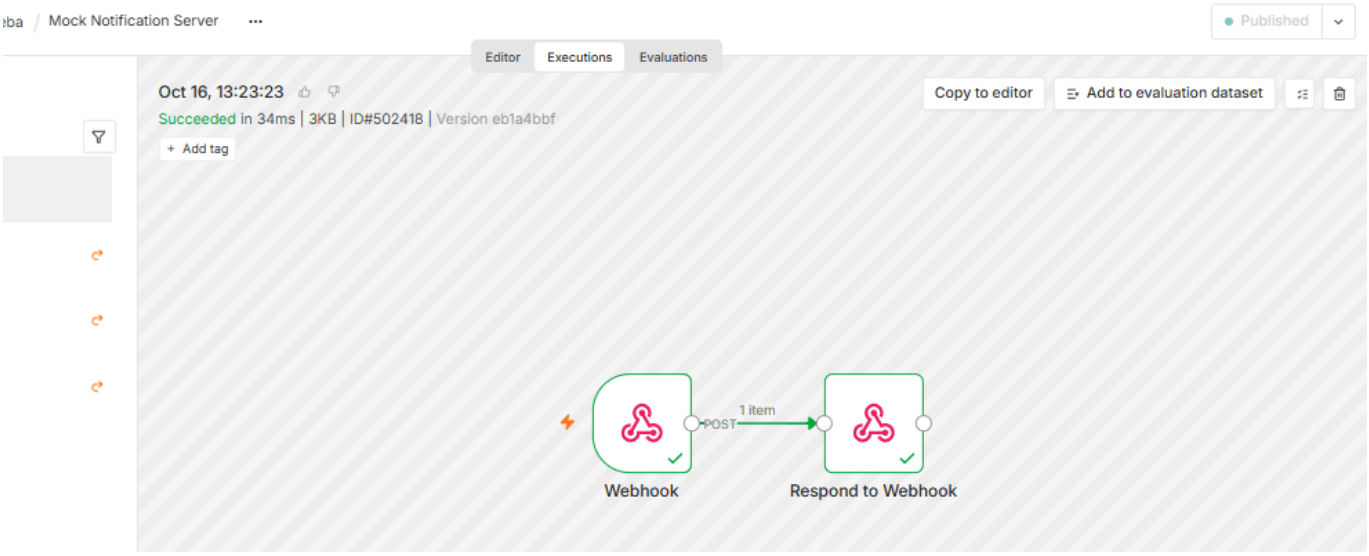


Explanation

A ticket marked Critical, which is outside the four configured priority values, is directed to the fallback route. The Set Email Fallback node assigns the Manual Triage Desk and prepares a review notification.

Source screenshot: image(20261016-144733).png

04 | Published mock notification server

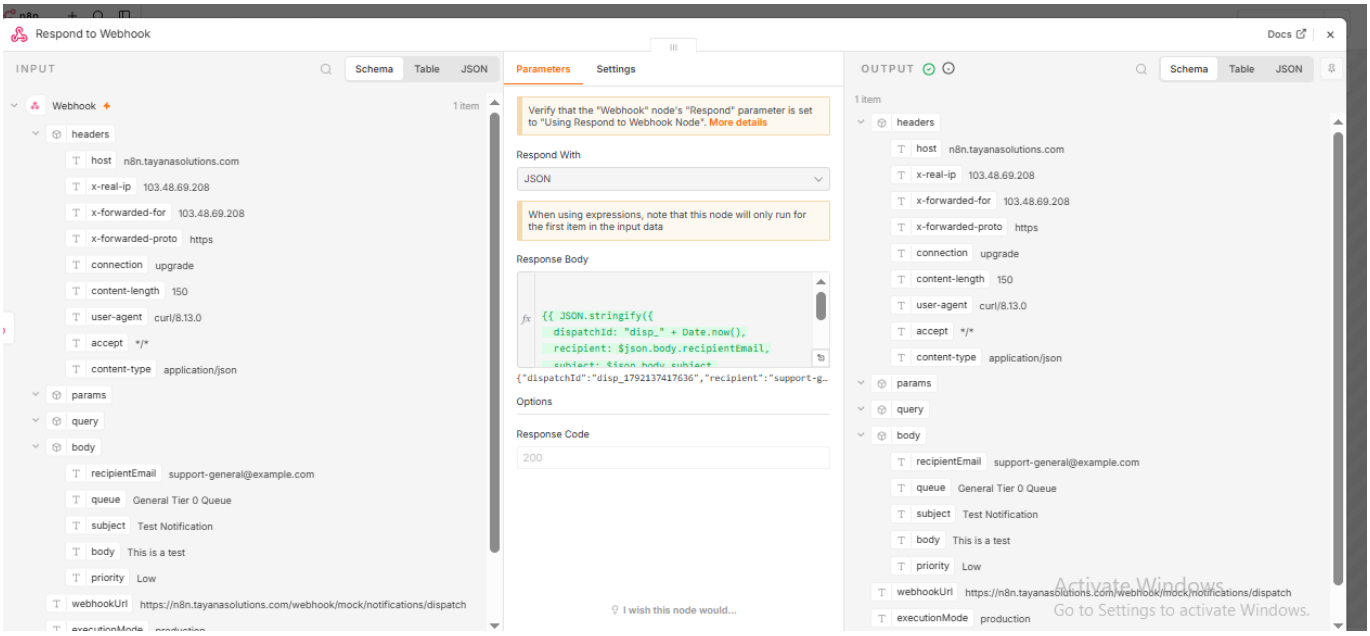


Explanation

The separate Mock Notification Server workflow is published and shows a successful execution. Its Webhook receives one POST item, and Respond to Webhook completes successfully.

Source screenshot: image(20261016-144745).png

05 | Dynamic mock server response

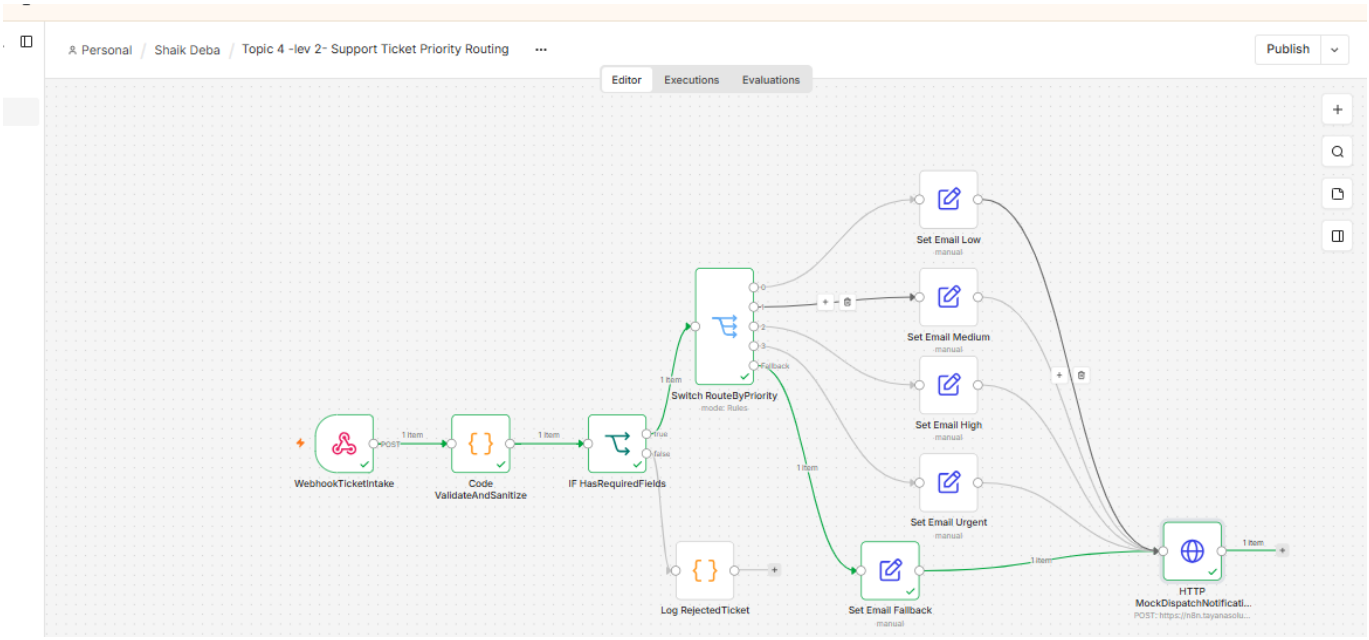


Explanation

The Respond to Webhook node uses a dynamic JSON expression to create a dispatch identifier and return the recipient, subject, queue and simulated delivery status. The response code is set to 200.

Source screenshot: image(20261016-144800).png

06 | Main workflow dispatch path

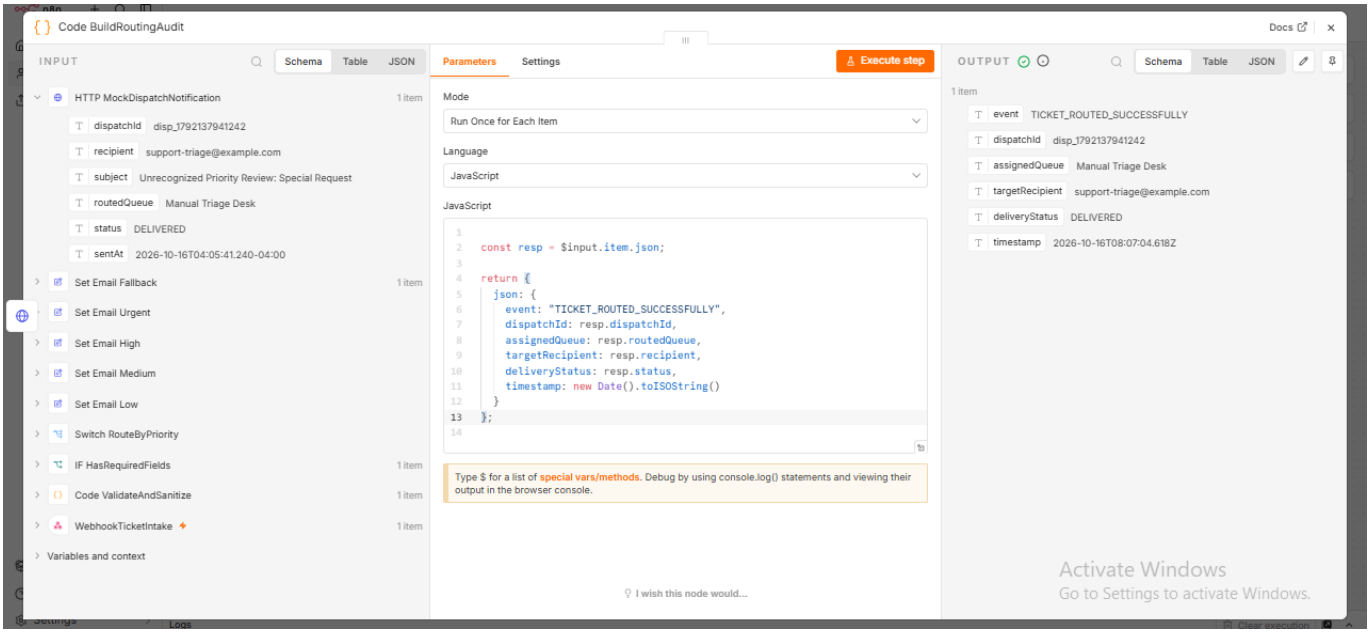


Explanation

The main workflow shows Webhook intake, validation, required-field checking, priority-based branching and the shared HTTP mock notification dispatch. The screenshot captures a successful Fallback dispatch with one processed item.

Source screenshot: image(20261016-144808).png

07 | Routing audit output

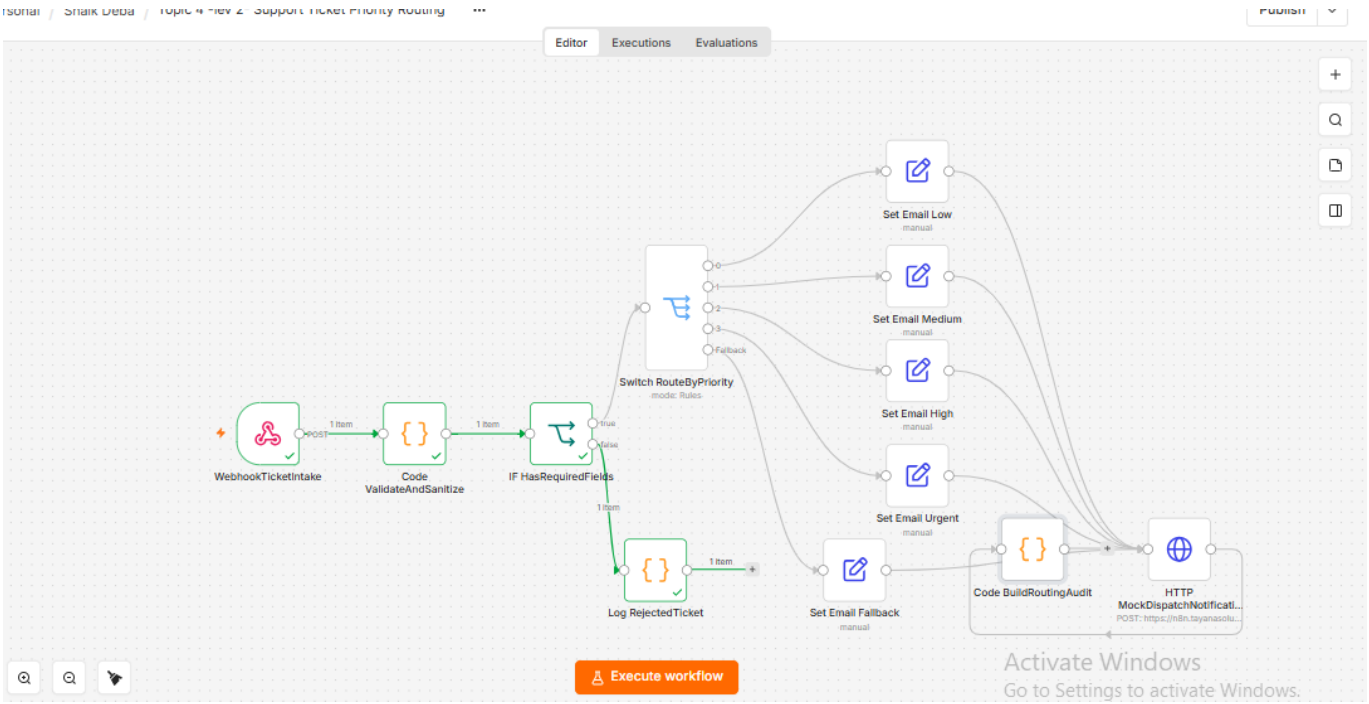


Explanation

The Code BuildRoutingAudit node records TICKET_ROUTED_SUCCESSFULLY. Its output confirms the Manual Triage Desk, support-triage@example.com recipient, dispatch identifier, simulated DELIVERED status and timestamp.

Source screenshot: image(20261016-144815).png

08 | Invalid ticket rejection path

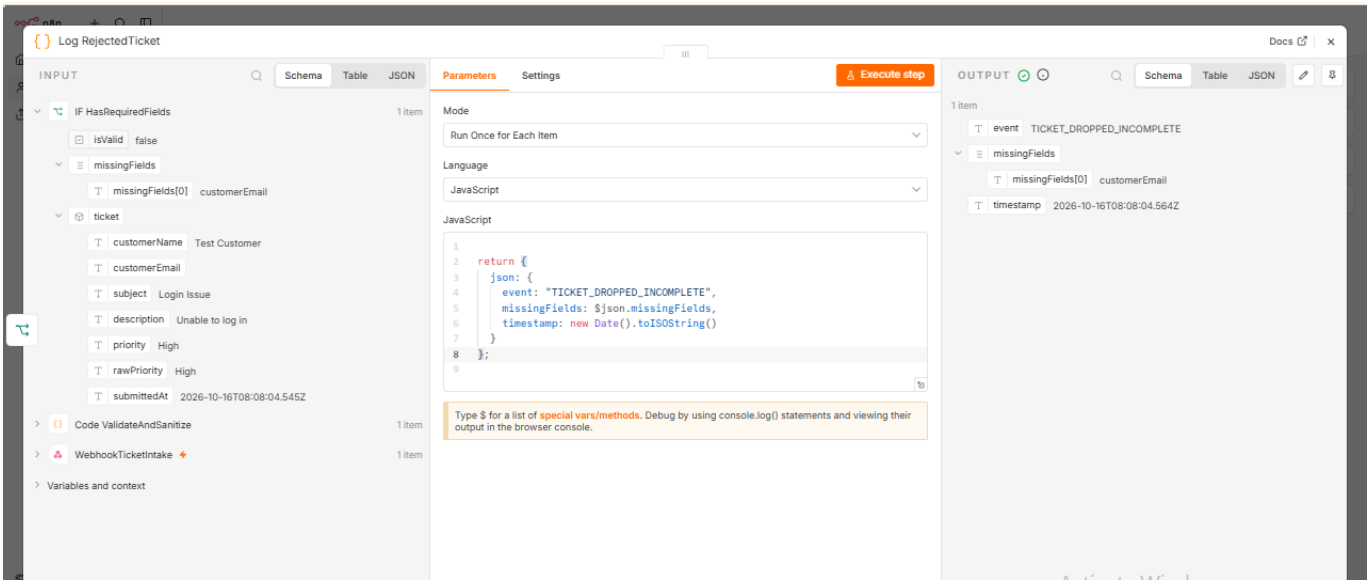


Explanation

A test ticket without customerEmail follows the false branch of IF HasRequiredFields to Log RejectedTicket. The screenshot shows one item reaching the rejection node rather than the notification route.

Source screenshot: image(20261016-144836).png

09 | Rejection audit details



Explanation

Log RejectedTicket produces the event TICKET_DROPPED_INCOMPLETE and identifies customerEmail as the missing required field. The output also contains a timestamp, confirming that the rejection was recorded.

Source screenshot: image(20261016-144844).png